MEDHAVA

Deployment — running the platform

This describes a design. It is how the platform is deployed and run once it is built. Nothing here claims to already be running.

It is written for whoever operates the platform. A business *using* the platform deploys nothing — it signs up in a browser. If you are looking for how a customer gets set up, that is the tenant guide, and it contains no commands at all.

What this document deliberately does not contain

An earlier version of it opened with **verify a Meta business account** and ended with **connect a WhatsApp provider**. Both were wrong here, and wrong in an instructive way.

Those are a **customer's** accounts. A business's conversations with its own customers belong to that business. The platform holds no messaging account, no marketplace seller account and no payment account of its own — it provides the place for a customer to plug theirs in. Nobody deploying this platform needs any of them, and a deployment document that starts there is answering a question nobody asked.

The one rule this document obeys

No layer here is welded to one supplier. Every choice below names what it is, and what else would do. The application is packaged as an ordinary container with nothing host-specific inside it, which is the single decision that keeps every other option open.

If moving the platform to a different host is ever hard, something host-specific has leaked in, and that is a bug rather than a fact of life.

1 · What has to exist before anything is deployed

What	Default	Also works
Somewhere to run containers	A virtual server you control	A managed container platform · a machine in your own building
A database	PostgreSQL	A managed Postgres service · Postgres you run yourself
A place for files	An S3-compatible object store	Any other S3-compatible provider · a self-hosted one · server disk with an off-box copy
A domain, with DNS you can edit	Any registrar	Any other — nameservers can be pointed anywhere
A certificate	Let's Encrypt, renewed automatically	Any certificate authority

Prices and free-tier limits change every few months, so none is quoted here. Check them at the source before committing money — a figure copied into a document is a figure somebody budgets from a year later.

2 · Secure the machine before anything listens on it

In this order, and the order matters.

1. Create a normal user with administrator rights. Stop using the root account.
2. Put your SSH key on it and confirm you can sign in with it.
3. **Open a second terminal and confirm key sign-in works there too.** Only then turn password sign-in off. If the key is wrong and you have already closed your only working session, you are locked out of your own machine.
4. Allow only the ports you actually use — the SSH port, and the two web ports. Deny the rest.
5. Turn on automatic security updates.
6. Install something that blocks repeated failed sign-in attempts.

Done when: password sign-in is off, you are certain you can still get in, and the machine is answering on nothing you did not intend.

3 · Give it room to breathe

Add swap space — space on disk the machine can use when memory runs short. Not for speed. It is so that one service having a bad minute cannot cause the machine to kill another one outright.

Done when: swap shows up in the machine's memory report.

---

4 · Point the names at the machine

Four names, all pointing at the same machine, each serving something different.

Name	Serves
the bare domain and `www`	The public site
`app`	The application, behind sign-in
Whatever you use for internal tools	Internal only, never public

Mail is separate and should stay separate: point mail records at whoever provides your mailboxes, not at this machine. Running your own mail server is a full-time job that has nothing to do with this platform.

Done when: each name resolves to the machine, checked from a connection that is not yours.

---

5 · Put a web server in front, and get certificates

The web server accepts connections, holds the certificates, and passes requests to the application. Keeping it separate from the application means the application never has to know about certificates, ports or redirects.

Configure it to refuse plain unencrypted connections, and to renew certificates by itself.

Done when: every name loads over an encrypted connection, and renewal has been tested rather than assumed.

---

6 · Release without anybody noticing

The rules that make a release boring:

- **Build the container once**, and move that exact container between environments. Rebuilding per environment means the thing you tested is not the thing you released.
- **Upload to a temporary name, then move it into place.** A visitor mid-request never sees a half-written file.
- **Keep the previous version ready.** Going back should be one command, and it should have been practised before anybody depends on it.
- **Run the checks before the release, not after.** A check that runs after is a report, not a gate.

Done when: a release can be done in the middle of a working day without anybody being warned, and undone just as quickly.

6a · The database role the application connects as

This is the single line that decides whether row-level security does anything at all.

The schema creates a role called `authenticated` — `NOLOGIN NOSUPERUSER` — and every isolation policy is written `FOR ALL TO authenticated`. The application must connect as a login role that inherits it, and that role must be **neither a superuser nor the owner of the tables**.

That is not a style preference. It was measured against a real Postgres in `core/tests/live.test.js`:

Connected as	Rows visible when one company is set
superuser / table owner	**both companies** — the policy is never consulted
the same, after `ALTER TABLE … FORCE ROW LEVEL SECURITY`	**both companies** — FORCE does not stop a superuser
`authenticated`	one — the policy applies

So a deployment that connects as the `postgres` superuser has every policy in the schema and no isolation whatsoever, and nothing about the running system would look wrong.

```
# create the login role, grant it the policy role, and hand the tables to somebody else
CREATE ROLE medhava_app LOGIN PASSWORD '…';   -- set from your secret store, never from this file
GRANT authenticated TO medhava_app;
```

Own the tables with a separate migration role. The application's connection string uses `medhava_app`, and every request sets `app.current_tenant` and `app.current_company` for the session before it touches a business table. An unset value is refused rather than matching everything — by one of two mechanisms, depending on how it came to be unset. A company set to the empty string is caught by the policy's explicit guard. A company that was never set, or was `RESET`, is NULL; `NULL <> ''` is NULL rather than false, so the guard does not short-circuit, the `::uuid` cast is reached, and Postgres raises. Both are fail-closed and no row escapes either way. `core/tests/live.test.js` asserts which one actually happens against a running database, so tightening the guard is a deliberate change with a test to update rather than a silent one.

---

7 · Settings and keys

Every key, password and connection string lives outside the code, in settings the service reads at startup.

- Readable only by the account the service runs as.

- Never committed. Not once, not temporarily — a key committed once is in every copy of that history forever.
- Different values per environment, so a practice copy can never reach real data.

Nothing in this platform ever asks anyone for a marketplace, bank or account password. Every outside connection uses a key the customer creates and can withdraw. That is a promise the product makes, and it holds here too.

Done when: a search of the entire history finds no key, and that search runs automatically on every change.

---

8 · Backups, and proving them

- Back up the database on a schedule, and copy it **off the machine**. A backup that lives only on the machine it protects is not a backup.
- Back up the settings and the web server configuration too. Restoring data onto a machine nobody can reconfigure is half a recovery.
- **Restore one.** Into a scratch environment, deliberately, before anybody needs it. An untested backup is a belief.

Done when: a restore has actually been done and checked, and it is repeated on a schedule.

---

9 · Watching it

Three questions, answerable without guessing:

Question	What answers it
Is it up?	An uptime check from outside your own network
Is it broken?	Error reports, grouped, with enough detail to act on
Is it slow, and where?	Timing recorded per operation

Keep the format standard so the tool reading it can be replaced without changing what the platform emits.

Done when: a failure can be traced from a user's click to the operation that failed, without adding new logging first.

---

10 · Environments

At least two, and they must not share anything.

	Practice	Live
Data	Realistic, never real	Real
Who can reach it	The team	Customers
Keys	Its own, valueless	Its own, guarded

Done when: a change can be taken end to end somewhere no customer can see, and nothing in the practice copy can reach anything real.

11 · The health check

Whenever something feels wrong, in this order:

1. Does the public site answer?
2. Does the application answer, and does it correctly refuse an unauthenticated request?
3. Are the services running?
4. How much memory and swap is in use, under real load?
5. Is the database reachable, and how long is it taking to answer?

The fourth is the one to watch on a small machine. Swap touched occasionally is fine. Swap in constant use means something is too big for the machine — and the fix is a bigger machine or a smaller workload, not patience.

What this costs

The machine	Depends on size and provider — check current pricing
The database	Free tiers exist and are real; check current limits
File storage	Charged by what you store and what you serve
Domain and certificates	The domain is yearly; certificates are free
Anything a customer connects	The customer's own account, and the customer's own cost

No figure is quoted from memory. Every one of these changes, and a stale price in a document is worse than no price, because somebody plans around it.

Every technical word used here, in plain language

Written for somebody who administers a server, which is not the same as somebody who has met every word on this page before. Only the terms this document actually uses are listed — padding it with definitions of words that never appear would make a count look better and the page worse.

17 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

platform

One piece of software that many separate businesses use at the same time, each seeing only its own information.

Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.

tenant

One business using the platform. Its people, its data and its settings are its own.

Us building mein ek office. Aapka office, aapka saamaan, aapka taala.

database

Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.

Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.

table

One kind of information inside the database — all your customers in one, all your orders in another.

Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

schema

The written plan of what information the system keeps and how the pieces connect.

Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.

row-level security

A lock inside the database itself, so one business physically cannot read another business's records — even if the software above it has a bug.

Taala darwaze pe nahin, tijori pe. Guard so bhi jaaye toh bhi tijori band rehti hai.

migration

A recorded change to the shape of the database, so every copy of the system can be updated the same way, in the same order.

Naksha badla toh likh ke rakha — taaki har site pe wahi badlav, usi tarike se ho.

backup

A copy of everything, kept somewhere else, so a mistake or a failure does not lose your work.

Zaroori kaagzaat ki photocopy, doosri jagah rakhi hui. Asli jal jaaye toh bhi kaam nahin rukta.

storage

Where files are kept — photographs, invoices, scanned documents. Different from the database, which keeps information rather than files.

Almari ke bagal wala godown. Register almari mein, par bade dabbe aur photo godown mein.

queue

A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.

Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.

job

One piece of work taken off the queue and done in the background.

Line mein se uthayi gayi ek parchi, ab uska kaam ho raha hai.

environment

A separate running copy of the system — one for trying things, one that customers actually use.

Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.

deployment

Putting a new version of the software in place so people start using it.

Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.

uptime

How much of the time the system is actually working and reachable.

Dukaan mahine mein kitne din khuli rahi. Band rahi toh customer wapas chala gaya.

provider

A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery.

Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.

role

What a person is allowed to see and do — a manager sees more than a counter staff member.

Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.

---

*This document describes how the platform is run. How a business gets set up **on** it is the tenant guide — which contains no commands, because that reader has no terminal.*
